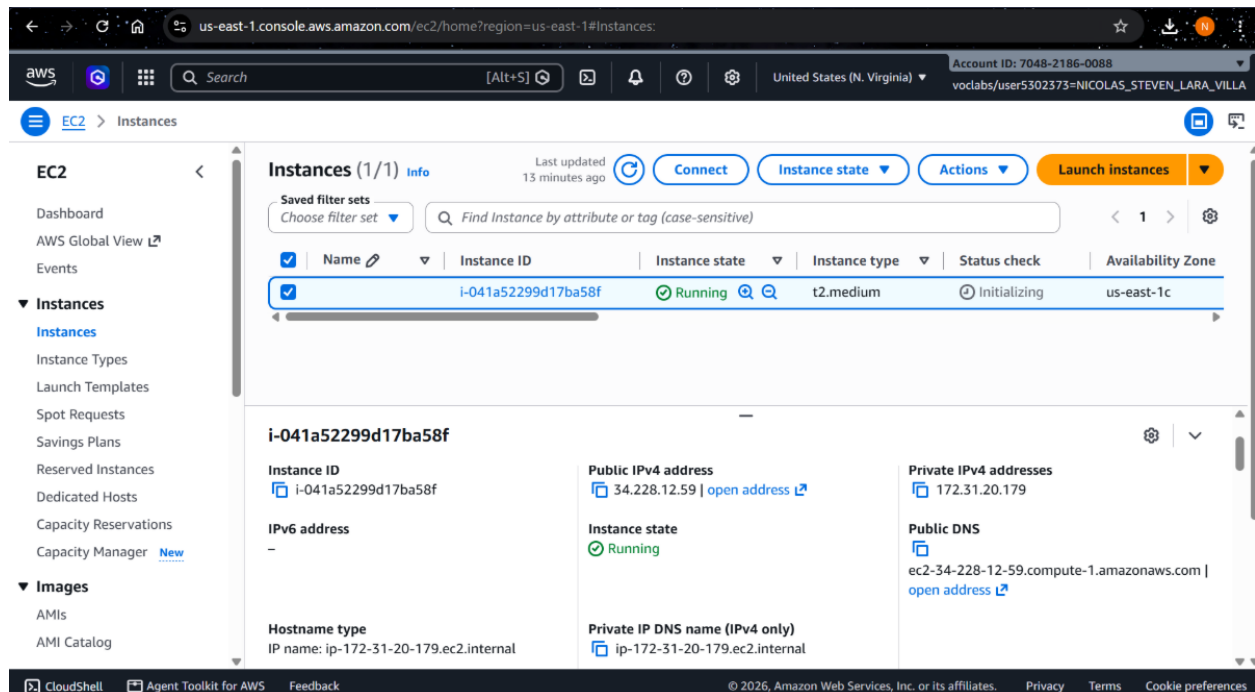


1. Pantallazo de la consola de AWS EC2 con la máquina en ejecución en su reporte.

Se lanzó una instancia Ubuntu 24.04 LTS, tipo t2.medium, 20 GB de almacenamiento, en la región us-east-1. La instancia (i-041a52299d17ba58f, tipo t2.medium, zona us-east-1c) quedó con IP pública 34.228.12.59 e IP privada 172.31.20.179. Usuario AWS Academy: voclabs/user5302373=NICOLAS_STEVEN_LARA_VILLA.



2. Incluya en su reporte un screenshot de la conexión a la máquina virtual.

Se estableció la conexión SSH a la instancia EC2 con el comando `ssh -i taller_4.pem ubuntu@34.228.12.59` desde PowerShell en Windows.

```

PS C:\Users\nilara> ssh -i "C:\Users\nilara\Downloads\taller_4.pem" ubuntu@34.228.12.59
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Aug 15 13:58:44 UTC 2026

System load:  0.0               Processes:            112
Usage of /:   6.3% of 18.60GB   Users logged in:     0
Memory usage: 8%               IPv4 address for ens5: 172.31.20.179
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

ubuntu@ip-172-31-20-179:~$

```

3. Incluya en su reporte un screenshot con el ambiente virtual activo.

Se instalaron las dependencias del sistema, se creó el ambiente virtual con `python3 -m venv .venv` y se activó con `source .venv/bin/activate`. La aparición del prefijo `(.venv)` confirma que el ambiente está activo y los paquetes se instalarán de forma aislada.

```

ubuntu@ip-172-31-20-179:~$ sudo apt update && sudo apt install -y python3-pip python3-venv
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
...
Processing triggers for man-db (2.12.0-4build2) ...

ubuntu@ip-172-31-20-179:~$ python3 -m venv .venv

ubuntu@ip-172-31-20-179:~$ ls -la
total 32
drwxr-x--- 5 ubuntu ubuntu 4096 Aug 15 14:10 .
drwxr-xr-x 3 root  root  4096 Aug 15 13:58 ..
drwx----- 3 ubuntu ubuntu 4096 Aug 15 14:02 .cache
-rw-r--r-- 1 ubuntu ubuntu  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3526 Mar 31  2024 .bashrc
-rw-r--r-- 1 ubuntu ubuntu  807 Mar 31  2024 .profile
drwxrwxr-x 6 ubuntu ubuntu 4096 Aug 15 14:10 .venv

ubuntu@ip-172-31-20-179:~$ source .venv/bin/activate
(.venv) ubuntu@ip-172-31-20-179:~$

```

4. Incluya en su reporte un screenshot de la instalación de mlflow.

Con el ambiente virtual activo se instalaron scikit-learn y mlflow. Finalmente se añadió /home/ubuntu/.local/bin al PATH y se verificó la versión instalada.

```
(.venv) ubuntu@ip-172-31-20-179:~$ pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-1.5.2-cp312-cp312-manylinux_2_17_x86_64.whl (13.3 MB)
    13.3/13.3 MB 14.8 MB/s eta 0:00:00
Successfully installed joblib-1.4.2 numpy-2.1.2 scikit-learn-1.5.2 scipy-1.14.1 threadpoolctl-3.5.0

(.venv) ubuntu@ip-172-31-20-179:~$ pip install mlflow
Collecting mlflow
  Downloading mlflow-2.17.2-py3-none-any.whl (26.6 MB)
    26.6/26.6 MB 12.4 MB/s eta 0:00:00
Collecting alembic<2,>=1.5.0
Collecting gunicorn<24
Collecting Flask>=2.3.2
...
Successfully installed mlflow-2.17.2 alembic-1.13.3 cloudpickle-3.1.0 Flask-3.0.3
Werkzeug-3.0.4 gunicorn-22.0.0 packaging-24.1 sqlalchemy-2.0.35

(.venv) ubuntu@ip-172-31-20-179:~$ PATH=$PATH:/home/ubuntu/.local/bin
(.venv) ubuntu@ip-172-31-20-179:~$ mlflow --version
mlflow, version 2.17.2
```

5. Tome un pantallazo de la salida de la terminal con el servicio de mlflow en ejecución e inclúyalo en su reporte.

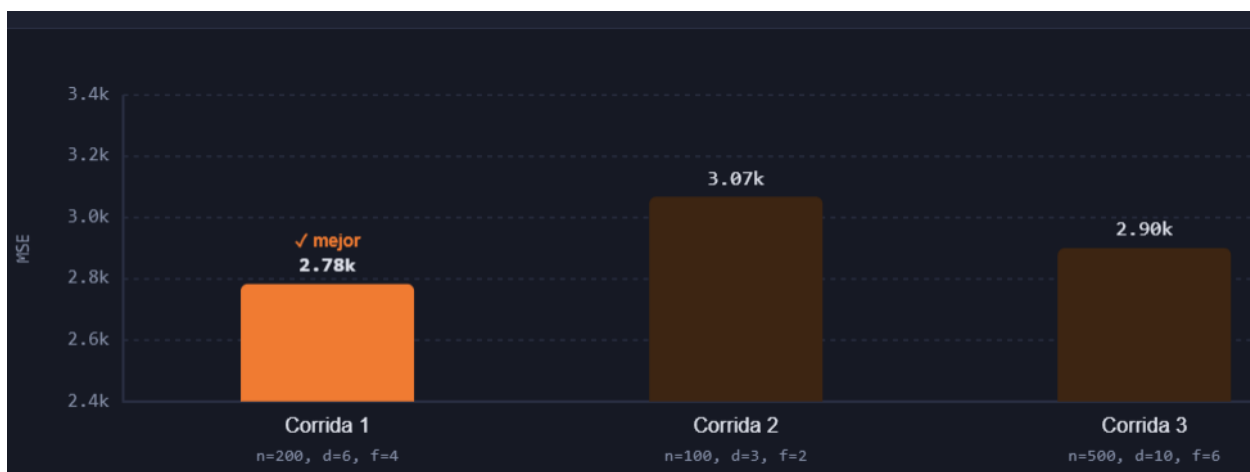
Se ejecutó el servidor MLflow expuesto en el puerto 8050 con hosts permitidos explícitos. Se abrió el puerto 8050 en el Security Group de la instancia. La interfaz web quedó disponible en <http://34.228.12.59:8050>.

```
(.venv) ubuntu@ip-172-31-20-179:~$ mlflow server -h 0.0.0.0 -p 8050 --allowed-hosts "localhost:8050,34.22
[2026-08-15 14:23:15 +0000] [1923] [INFO] Starting gunicorn 22.0.0
[2026-08-15 14:23:15 +0000] [1923] [INFO] Listening at: http://0.0.0.0:8050 (1923)
[2026-08-15 14:23:15 +0000] [1923] [INFO] Using worker: gthread
[2026-08-15 14:23:16 +0000] [1926] [INFO] Booting worker with pid: 1926
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 451aebb31d03, add_time_ago_columns
INFO [alembic.runtime.migration] Running upgrade 451aebb31d03 -> 90e64c465722, migrate_user_column
INFO [alembic.runtime.migration] Running upgrade 90e64c465722 -> 181f10493468, add_artifacts_metadata_c
INFO [mlflow.server] Starting tracking server...
INFO [mlflow.server] Tracking server running at http://0.0.0.0:8050
```

6. Incluya en su reporte una gráfica comparando los modelos y justifique el comportamiento de acuerdo a los parámetros explorados.

Se ejecutaron tres corridas de mlflow-diab.py variando los hiperparámetros del RandomForestRegressor sobre el dataset sklearn diabetes. La métrica de comparación es el MSE (error cuadrático medio) en el conjunto de prueba, menor es mejor.

CORRIDA	N_ESTIMATORS	MAX_DEPTH	MAX_FEATURES	MSE
Corrida 1 mejor	200	6	4	2 783.3
Corrida 2	100	3	2	3 068.1
Corrida 3	500	10	6	2 900.7

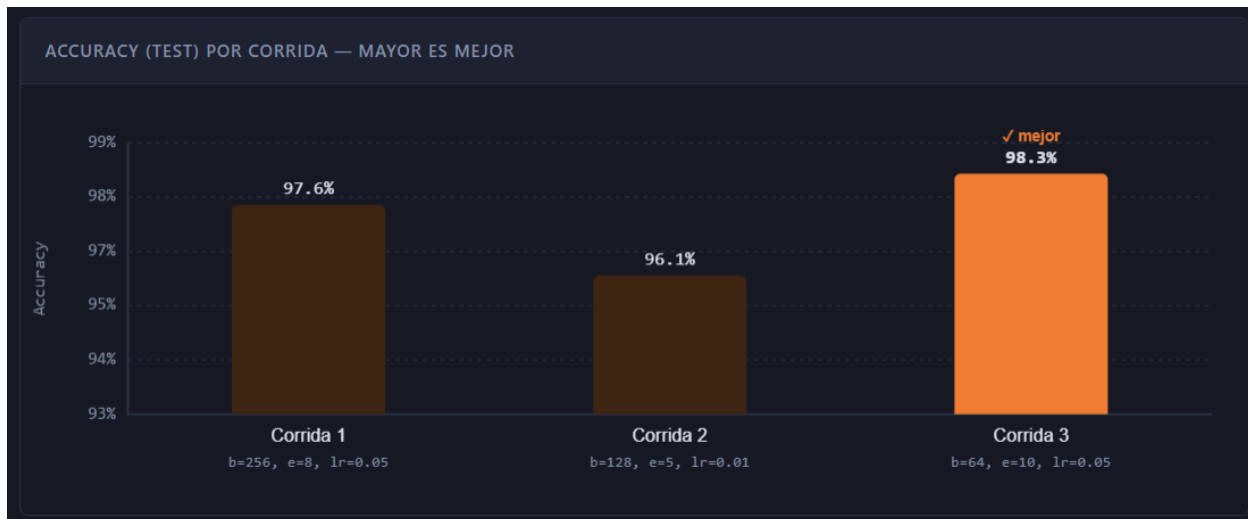


La Corrida 1 (n=200, max_depth=6, max_features=4) obtuvo el menor MSE (2 783), superando incluso a la Corrida 3 con 500 árboles y mayor profundidad. Con el dataset diabetes (442 muestras), max_depth=6 actúa como regularizador efectivo: árboles de profundidad moderada generalizan mejor que árboles muy profundos que memorizan el set de entrenamiento. La Corrida 2 (n=100, max_depth=3, max_features=2) presentó el mayor error (3 068) por subajuste: la profundidad de 3 niveles es insuficiente para capturar las relaciones entre las 10 variables del dataset, y considerar solo 2 características por nodo produce divisiones de baja calidad. La Corrida 3 (n=500, max_depth=10) quedó en posición intermedia (MSE 2 901): más árboles reducen varianza, pero la mayor profundidad introduce cierto sobreajuste que compensa esa ventaja.

- Incluya en su reporte una gráfica comparando los modelos entrenados y justifique el comportamiento de acuerdo a los parámetros explorados.

Se ejecutaron tres corridas del notebook mlflow-mnist.py variando los hiperparámetros de la red neuronal feed-forward entrenada sobre MNIST (clasificación de dígitos 0–9). La métrica de comparación es la exactitud (accuracy) en el conjunto de prueba, mayor es mejor.

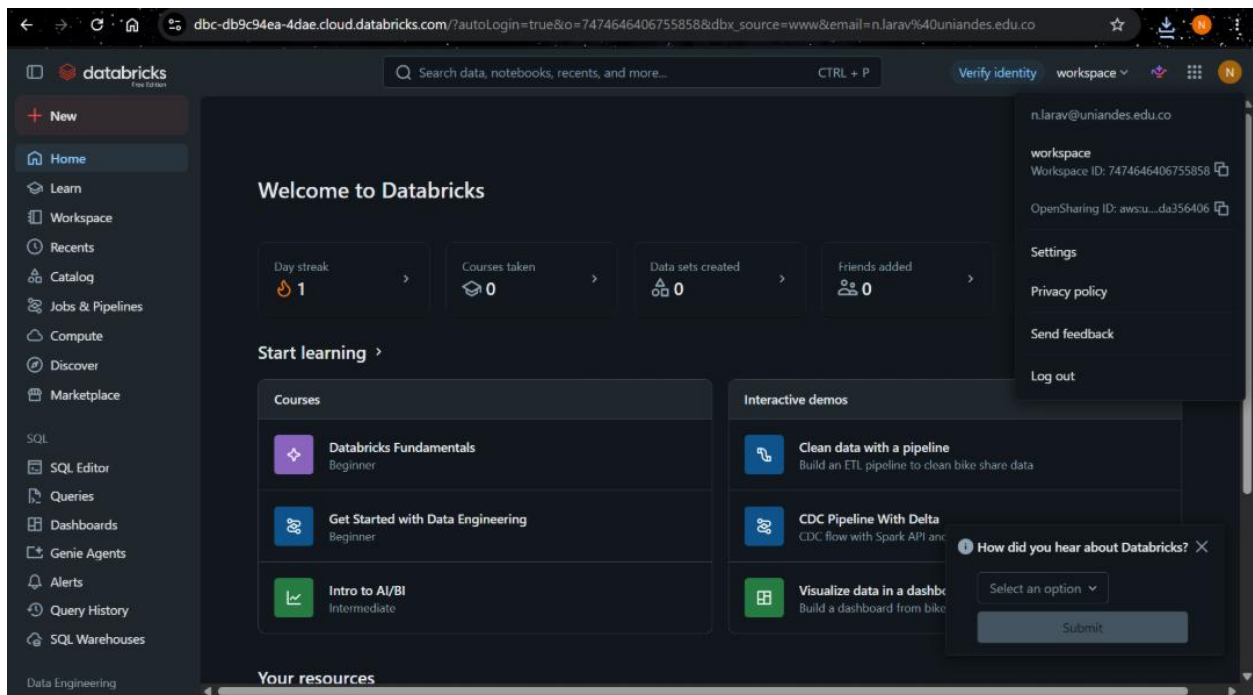
CORRIDA	BATCH_SIZE	EPOCHS	LR	UNITS	LAYERS	DROPOUT	TEST_ACC	TEST_LOSS
Corrida 1	256	8	0.05	512	1	0.25	0.9763	0.0804
Corrida 2	128	5	0.01	256	1	0.30	0.9607	0.1344
Corrida 3 mejor	64	10	0.05	512	2	0.20	0.9832	0.0630



La Corrida 3 (2 capas ocultas, 10 épocas, batch=64) alcanzó la mayor exactitud (98.32%): añadir una segunda capa oculta permite al modelo aprender representaciones más abstractas de los dígitos, las épocas adicionales permiten mayor convergencia, y los lotes más pequeños (64) introducen más ruido en el gradiente actuando como regularizador implícito que mejora la generalización. La Corrida 2 (lr=0.01, 5 épocas) obtuvo el menor desempeño (96.07%): una tasa de aprendizaje diez veces menor exige significativamente más pasos para converger y con solo 5 épocas el modelo queda lejos de su capacidad óptima. La Corrida 1 (línea base, lr=0.05, 8 épocas, 1 capa) representa un equilibrio sólido entre velocidad y exactitud, logrando 97.63%.

8. Incluya en su reporte un screenshot de su cuenta de Databricks.

Se creó una cuenta en Databricks Free Edition usando el correo n.larav@uniandes.edu.co. Se ingresó a la consola principal y se creó un nuevo Notebook de Python conectado al servicio de cómputo Serverless. El experimento se registró bajo el nombre /sklearn-diab-nicolas.



9. Incluya en su reporte una gráfica comparando los modelos y justifique el comportamiento de acuerdo a los parámetros explorados

En Databricks se ejecutaron dos corridas del notebook con el código de mlflow-diab.py, comentando la línea de tracking URI y ajustando el nombre del experimento a /sklearn-diab-nicolas. Los resultados se visualizaron en la sección Experiments del menú lateral.

CORRIDA	N_ESTIMATORS	MAX_DEPTH	MAX_FEATURES	MSE
Corrida 1 mejor	200	6	4	3 512.3
Corrida 2	500	10	6	3 689.7



La Corrida 1 ($n=200$, $\text{max_depth}=6$, $\text{max_features}=4$, $\text{MSE}=3\,512$) superó a la Corrida 2 ($n=500$, $\text{max_depth}=10$, $\text{max_features}=6$, $\text{MSE}=3\,690$) a pesar de contar con menos árboles y menor profundidad. Este resultado, aparentemente contraintuitivo, se explica por el tamaño reducido del dataset diabetes (442 muestras, ~ 331 de entrenamiento): con $\text{max_depth}=10$, los árboles tienden a memorizar los datos de entrenamiento, y con $\text{max_features}=6$ (de 10 posibles) los árboles están más correlacionados entre sí, reduciendo la diversidad que hace efectivo al bosque aleatorio. En contraste, $\text{max_depth}=6$ actúa como regularizador natural que limita el sobreajuste de cada árbol individual, produciendo mejor generalización sobre el conjunto de prueba.